



MERGULHO TÉCNICO

Geti IoT / HydroSense

Arquitetura, firmware, protocolo de dados e decisões de engenharia do monitoramento e controle de hidroponia com IoT.

ESP32 — firmware C++

Blynk Cloud — datastreams

Dashboard web — HTML + HTTP API

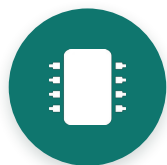
Hidroponia já foi vista no seminário anterior — aqui é arquitetura e código.

Gabrielle Oliveira Santana · Gabriel Sandrini · Igor Murici



Arquitetura em 3 camadas

02



BORDA ESP32 — firmware C++

Lê 5 sensores, decide e aciona 4 atuadores. DHT22 real; nível, pH e EC são potenciômetros (Wokwi); atuadores são LEDs.



NUVEM Blynk Cloud

Plataforma gerenciada. Expõe os dados como datastreams em pinos virtuais, acessíveis por uma API HTTP REST.



CLIENTE HydroSense (index.html)

SPA de um único arquivo HTML. Fala direto com a API HTTP — sem backend, sem servidor de aplicação.

publica a cada 2 s →

← consulta a cada 5 s

Cadências assimétricas. São relógios independentes, sem handshake — e isso só funciona porque a decisão não depende do app estar sincronizado.



Mapa de dados — pinos virtuais (V0–V10)

03

Pino	Conteúdo	Tipo	Direção
V0–V1	temperatura, umidade	Double	device → nuvem
V2–V4	nível, pH, EC	Double	device → nuvem
V5–V8	nutriente, ácido, base, válvula	Integer 0/1	device → nuvem
V10	limites de controle (JSON)	String	app → device

O contrato de dados

A interface entre device e app é o conjunto de pinos — um **contrato de dados**.

Sensores e atuadores fluem **device → nuvem**; só os limites fluem **app → device**.

O device é o **único escritor** dos atuadores (V5–V8); o app só lê.

A **assimetria de quem-escreve-o-quê** é o que define a arquitetura inteira — e é o assunto dos próximos slides.



Decisão #1 — a inteligência fica na borda

04

```
void sendSensors() {  
  float level = analogRead(PIN_LEVEL)/4095.0*100.0;  
  float ph     = analogRead(PIN_PH)/4095.0*14.0;  
  float ppm    = analogRead(PIN_EC)/4095.0*2000.0;  
  Blynk.virtualWrite(V2, level); /* V3, V4 ... */  
  
  autoActuate(ph, ppm, level); // o device decide,  
                               // todo ciclo  
}
```

A lógica de decisão roda no **firmware**, dentro do loop de 2 s — não no app.

Defaults seguros embarcados (phLo=5.5 ... tankFull=95) → autorregula no 1º boot.

Funciona sem internet e sem app aberto → o app não é dependência operacional.

Alternativa rejeitada: *app decide e comanda* → cai a rede, a planta fica sem controle.

Reduz a superfície de falha: o caminho crítico não passa pela internet.



O laço de controle — ler → decidir → atuar

05

```
void autoActuate(float ph, float ec, float lvl){  
  if (ph > phHi + phWarn)  
    setActuator(PIN_ACID, stAcid, true, ...);  
  else if (ph <= phHi)  
    setActuator(PIN_ACID, stAcid, false, ...);  
  // pH baixo->base; EC baixo->nutriente (idem)  
}  
  
void setActuator(pin,&st,on,vpin,name,why){  
  if (st == on) return; // change guard  
  st = on; digitalWrite(pin, on);  
  Blynk.virtualWrite(vpin, on?1:0); // espelha  
}
```

setActuator centraliza atuar + espelhar na nuvem + logar o motivo ([ACT]).

Change guard (if state==on return): evita chatter e quebra o eco virtualWrite → BLYNK_WRITE.

Regras: pH alto → ácido · pH baixo → base · EC baixo → nutriente.

Uma linha resolve dois problemas: sem chatter e sem laço infinito de escritas.



Histerese — a zona morta

06

Sem histerese: liga e desliga no mesmo ponto → atuador “pisca” perto do limite (desgaste).

Com histerese: liga em $phHi + phWarn$, desliga em $phHi$ → banda morta dá estabilidade.

Mesmo padrão para base ($phLo$) e nutriente ($ecLo$) — *Warn é a largura da banda.

```
// liga ácido no CRITICO,  
// desliga so ao VOLTAR a faixa ideal  
if (ph > phHi + phWarn)  
    setActuator(PIN_ACID, stAcid, true, ...); // liga  
else if (ph <= phHi)  
    setActuator(PIN_ACID, stAcid, false, ...); // desliga  
//      ^^^^^^^^^^^^^^^^^      ^^^^^  
//      limiar de liga != limiar de desliga  
//      banda morta = phWarn
```

É o mesmo princípio de um termostato: uma vez ligado, o atuador só relaxa após recuperação real.



Válvula multifunção & guarda anti-transbordo

07



```
// Ordem importa: a guarda e avaliada PRIMEIRO
if (level >= tankFull)
    setActuator(PIN_WATER,...,false,"tanque cheio");
else if (level < tankCrit)
    setActuator(PIN_WATER,...,true, "tanque baixo");
else if (ec > ecHi + ecWarn)
    setActuator(PIN_WATER,...,true, "EC alto (diluir)");
else if (level >= tankWarn && ec <= ecHi)
    setActuator(PIN_WATER,...,false,"ok");
```

Dupla função: reabastecer tanque baixo ou diluir EC alto.

Guarda anti-transbordo: level >= tankFull é o primeiro if → trava de segurança.

Prioridade no código = prioridade de risco: diluir nunca pode transbordar.

Fail-safe: a trava de segurança tem precedência sobre o objetivo operacional — e é literalmente a primeira linha da função.



Configuração em runtime — JSON no V10

08

Problema: mudar a faixa ideal sem regravar/recompilar o firmware.

Solução: todos os limites num único JSON atômico, no pino V10.

App é a fonte da verdade; device adota em 1 ciclo;
BLYNK_CONNECTED faz syncVirtual(V10).

1 string > 9 pinos: atômico, extensível e à prova de estado parcial.

```
BLYNK_WRITE(V10){           // app escreveu limites
  StaticJsonDocument<256> doc;
  if (deserializeJson(doc, param.asStr()))
    return;                 // JSON invalido -> mantem
  phLo = doc["phLo"] | phLo; // '| phLo' = fallback
  ecHi = doc["ecHi"] | ecHi; /* ... 9 limites ... */
}
// payload: {"phLo":5.5,"phHi":6.5,"ecLo":600,
//           "ecHi":1000,"tankFull":95, ...}
```

A diferença entre reconfigurar e reimplantar. Nova chave ≠ novo datastream; JSON parcial é seguro pelo fallback.



O cliente web — polling, read-only, fonte única

09



```
const POLL_MS = 5000;
async function poll(){
  const d = await (await fetch(
    `${BLYNK}/getAll?token=${TOKEN}`)).json();
  // V0-V4 -> cards/charts ; V5-V8 -> read-only
  // V10 -> form de limites
  Object.keys(ACT).forEach(k =>
    syncActuatorFromPin(k, num(ACT[k].pin)));
}
```

Lê tudo via `getAll` a cada 5 s; escreve só o V10 (limites).

Atuadores são read-only no app — mostram estado, não ligam/desligam.

Fonte única de comando (o device): evita dois “donos” disputando a mesma bomba.

SPA sem framework e sem build: fetch + DOM puro num arquivo.

Centralizar a atuação elimina uma classe inteira de conflito. Simplicidade é uma escolha de arquitetura.



Segurança — token, segredos e exposição



Único segredo — o Auth Token do device na Blynk.



Fora do código versionado — secrets.h (firmware) e env.js (app), ambos no .gitignore.



Repositório pode ser público — sem vaziar a credencial.



Limite honesto do protótipo

App roda no browser → token visível a quem tem a URL (?token= / localStorage). Aceitável numa aula; em produção, não.



Correção para produção

Um backend/proxy guarda o token no servidor; o front-end nunca o vê.

Reconhecer o risco e já ter a mitigação desenhada é, em si, boa engenharia.



Trade-offs técnicos

11

Decisão	Escolha	Ganha ↑ / Paga ↓
Onde decide	borda (firmware)	resiliência / update exige reflash
Config	JSON em runtime (V10)	flexibilidade / o que muda muito sai do firmware
Plataforma	Blynk gerenciada (buy)	velocidade / vendor lock-in
Hardware	simulação Wokwi	custo / iteração / fidelidade física
Cliente	SPA sem backend	simplicidade / token exposto



Escalabilidade. 1 template de pinos → N devices, sem reescrever firmware nem app. Cada estufa é só mais um device do mesmo template — é o que transforma protótipo em produto.



Demonstração & encerramento



1

Device é o cérebro

App fechado. Giro o pot de pH acima de $\text{pH}_{\text{Hi}} + \text{pH}_{\text{Warn}} \rightarrow [\text{ACT}]$ Bomba Acido $\rightarrow$ LIGADA (auto pH alto). LED acende, sem browser.



2

Fail-safe

EC alto liga a válvula (diluir). Subo o nível a $\text{tankFull} \rightarrow$ DESLIGADA (auto tanque cheio), mesmo com EC ainda alto.



3

Config em runtime

Mudo o EC máx em Ajustes $\rightarrow$ JSON no V10 $\rightarrow$ device loga [CFG] $\rightarrow$ novo limiar em 1 poll, sem reflash.

Serial Monitor: linhas [DATA] (leituras tagueadas OK/ALERTA/CRITICO) e [ACT] (com o motivo da decisão).

Controle distribuído: decisão na borda (resiliência), config em runtime (flexibilidade), cada dado com um dono único.

Obrigado!